
PsrPopPySuper Documentation

Release 1

D Tripathi

Jul 06, 2026

CONTENTS

1	Introduction	3
2	Installation	5
2.1	Download the package	5
2.2	Installing the package	5
2.3	Legacy Fortran Compilation (if needed)	6
3	Command-line scripts	7
3.1	populate.py	7
3.2	dosurvey.py	8
3.3	evolve.py	8
3.4	view.py	10
3.5	visualize.py	10
4	Tutorial - Basic Usage	11
4.1	Generate Population Model	11
4.2	Evolve a Population Model	11
4.3	Simulate a Pulsar Survey	12
4.4	Using PsrPopPySuper as a Python Module	12
5	Module-level Documentation	13
5.1	<i>pulsar</i> – Creates/stores a pulsar object	13
5.2	<i>population</i> – Creates/stores a population	13
5.3	<i>survey</i> – Read a survey file into a survey object	14
5.4	<i>populate</i> – Create a population object	14
5.5	<i>radialmodels</i> – Container for radial distn models	14
5.6	<i>galacticops</i> – Container for functions relating to the Galaxy	15
5.7	<i>evolve</i> – Evolve a pulsar population object	16
6	Indices and tables	17
	Python Module Index	19
	Index	21

Contents:

INTRODUCTION

PsrPopPySuper is a modernized Python3 port of Sam Bates' PsrPopPy code (<https://github.com/samb8s/psrpoppy>). This project retains the original code's functionality and most of the user facing APIs while updating interfaces and adding newer electron models like NE2025 and YMW16.

Contributions are welcome via the project's GitHub repository.

INSTALLATION

To get started with PsrPopPySuper there are a few steps you'll need to go through.

PsrPopPySuper is currently supported on Linux and Mac OS X, with python version 3.11 or later.

2.1 Download the package

The source for PsrPopPy can be downloaded from [GitHub](#) from the [PsrPopPy page](#). The source will contain the Python modules and scripts needed both for basic and advanced use.

2.2 Installing the package

PsrPopPy now supports standard Python package installation using pip. This automatically handles the Fortran compilation and installation.

2.2.1 Prerequisites

- Python 3.11 or later
- gfortran compiler (for Fortran library compilation)

On macOS, install gfortran using Homebrew:

```
brew install gcc
```

On Linux, install gfortran using your package manager:

```
# Ubuntu/Debian
sudo apt-get install gfortran

# CentOS/RHEL/Fedora
sudo yum install gcc-gfortran # or dnf install gcc-gfortran
```

2.2.2 Installation

With Conda

Use the environment.yml to make the conda environment and activate it:

```
conda create --file environment.yml --name PsrPopPySuper
conda activate PsrpoppySuper
```

Replace PsrPopPySuper with your own preferred name of the environment:

```
pip install -e .
```

Without Conda

Before moving on without conda ensure you have python version 3.11+ and have working c and fortran compiler:

```
cd PsrPopPySuper
pip install -r requirements.txt
pip install -e .
```

This will:

- Compile the Fortran and C libraries using gfortran and gcc
- Install the Python package
- Install command-line scripts (dosurvey, populate, evolve)

2.3 Legacy Fortran Compilation (if needed)

Although PsrPopPySuper is a Python-based package, some of the algorithms have been kept in their native FORTRAN for speed and ease of programming (e.g. the NE2001 electron model, coordinate conversion...). The pip installation now handles this automatically, but if you need to compile manually:

From the base directory:

```
cd psrpoppy/fortran
```

To use make, edit `makefile.<OSTYPE>` and ensure that the `gf` variable points to the location of your gfortran compiler. Then simply type `make`. All being well, four `.so` files will be generated.

Failing this, edit either `make_mac.sh` or `make_linux.sh`, depending upon your system, so that the `gf` variable points to your local gfortran/f77 compiler. Running the script:

```
bash make_<os>.sh
```

COMMAND-LINE SCRIPTS

3.1 populate.py

- n** <number of pulsars>
Required: Number of pulsars to generate; or to detect in a survey
- o** <output>
Output file name for population model (def=populate.model)
- asc** <ascii output file name>
Output file name for ascii output file (def=populate.txt)
- surveys** <SURVEY NAME(S)>
List of surveys to use when trying to detect pulsars (default=None)
- z** <scale height>
Scale height of pulsars about Galactic plane, in kpc (default=0.33)
- w** <width>
Pulse width to use when generating pulsars (default=0, use beaming model)
- si** <SImean SIsigma>
Spectral index mean and standard deviation (default=-1.6, 0.35)
- sc** <scatter index>
Spectral index of scattering law to use (default=-3.86, Bhat et al model)
- pdist** <distribution name>
Distribution type for pulse periods (default=lnorm)
Supported: 'lnorm', 'norm', 'cc97'
- p** <mean stddev>
Mean and standard deviation to use in period dist 'lnorm' or 'norm' (def=-2.7, 0.34)
- rdist** <radial model>
Model to use for Galactic radial distribution of pulsars
Supported: 'lfl06', 'yk04', 'isotropic', 'slab', 'disk'
- dm** <Electron model>
Model to describe the Galactic electron distribution
Supported: 'ne2025', 'ne2001', 'ymw16'

-gps <fraction 'a'>
Add <fraction> pulsars with GHz-frequency turnovers with index a

-doublespec <fraction alpha1>
Add <fraction> pulsars with low-frequency (below 1GHz) spectral index of alpha1

--nostdout
Turn off writing to stdout. Useful for many iterations eg. in large simulations

3.2 dosurvey.py

-f <filename>
Input population model to use (default=populate.model)

-surveys <SURVEY NAME(S)>
Required: Name(s) of surveys to simulate on the population model

--noresults
By default, a .results file is written, containing a model of the population detected in the survey. This option switches off the writing of this file.

--asc
Write the survey model in plain ascii (psrpop old style). Not recommended, since the cPickle '.results' file is easier to work with.

--summary
Write a short .summary file (per survey) describing number of detections, number of pulsars outside survey area, number smeared, and number not beaming

--nostdout
Turn off writing to stdout. Useful for many iterations eg. in large simulations

3.3 evolve.py

-n <number of pulsars>
Required: Number of pulsars to generate or detect

-o <output>
Output file name for population model (def=evolve.model)

-asc <ascii output file name>
Output file name for ascii output file (def=None)

-surveys <SURVEY NAME(S)>
List of surveys to use when trying to detect pulsars (default=None)

-dm <Electron model>
Model to describe the Galactic electron distribution
Supported: 'ne2025', 'ne2001', 'ymw16'

-tmax <max age>
Maximum initial age of pulsars, in years (def=1.0E9)

- p** <mean stddev>
Period distribution mean and standard deviation, seconds (def=0.3, 0.15)
- ldist** <distribution>
Distribution to use for luminosities
Supported: 'fk06', 'lnorm', 'pow'
- l** <parameters>
Luminosity distribution parameters (def=-1.5, 0.5)
- b** <mean stddev>
Mean and standard deviation of log-normal B field distribution (Gauss, def=12.65, 0.55)
- vmodel** <model>
Velocity model to use
Supported: 'gaussian', 'exp'
- v** <mean stddev>
Velocity distribution values (def=0, 180 km/s)
- si** <mean stddev>
Spectral index mean and standard deviation (def=-1.4, 0.96)
- spinmodel** <model>
Spin-down model to employ
Supported: 'fk06', 'cs06'
- alignmodel** <model>
Pulsar alignment model to use
Supported: 'orthogonal', 'random', 'rand45', 'wj08'
- aligntime** <time>
Alignment timescale (def=None)
- beammodel** <model>
Beaming model to use (def=tm98)
Supported: 'tm98', 'none', 'const', 'wj08'
- w** <width>
Pulse width to use when generating pulsars, percent (def=5.0)
- wmod** <width model>
Pulse width model to use
Supported: None, 'kj07'
- sc** <scatter index>
Spectral index of scattering law to use (def=-3.86)
- eff** <efficiency cutoff>
Efficiency cutoff value (def=None)
- z** <scale height>
Exponential z-scale height for the population, in kpc (def=0.05)

-bi <braking index>
Braking index value to use (def=0 = model default)

--nostdout
Turn off writing to stdout.

--nodeathline
Turn off the deathline.

--nospiralarms
Turn off spiral arms galactic distribution.

--keepdead
Keep dead pulsars in the population model.

3.4 view.py

-f <input model>
Select the population model to view (default=populate.model)

-p <param name>
Select the population parameter to plot
Supported: 'period', 'dm', 'gl', 'gb', 'lum', 'alpha', 'r0', 'rho', 'width', 'spindex', 'scindex', 'dist'

--logx
Plot log10 of the values

3.5 visualize.py

-f <model>
Select a population model to plot (default = populate.model)

-frac <F>
Plot a fraction <F> of the population for speed gains

TUTORIAL - BASIC USAGE

This page will outline a very simple pipeline for basic population simulations with PsrPopPy.

The following describe the simple command line workflow with the `psrpoppy` scripts.

4.1 Generate Population Model

Population models are generated using the `populate` script (installed automatically with pip). A common use would be to generate a population of normal pulsars using the Parkes Multibeam Pulsar Survey as a basis. This survey detected 1038 pulsars (at last count). Using default radial distribution, period and luminosity models, we can generate such a population using the command:

```
populate -n 1038 -surveys PMSURV
```

This will then run for a few minutes, until the model PMSURV survey has detected 1038 pulsars. The file `populate.model` will be produced by default, which is a `pickled` population object.

4.2 Evolve a Population Model

The `evolve` script takes a snapshot-style population model and evolves it according to the Ridley & Lorimer pulsar evolution method. This is useful when you want to generate a more realistic, age-dependent population before passing it on to a survey simulation.

A basic usage example is:

```
evolve -n 1000 -surveys PMSURV -dm ne2025 -o evolve.model
```

This command will generate an evolved population of pulsars using the NE2025 electron density model. The output file `evolve.model` can then be used by `dosurvey` or visualised with `view.py`.

The `evolve` script also supports the same survey and population controls as `populate`, including pulse width, beaming, spin-down, and electron model options. For example, to use a fixed 5% pulse width and the WJ08 alignment model, run:

```
evolve -n 1000 -surveys PMSURV -w 5.0 -alignmodel wj08 -o evolve.model
```

If you only want to generate the evolved population without checking a survey, omit the `-surveys` option and use the output file directly:

```
evolve -n 1000 -dm ne2025 -o evolve.model
```

4.3 Simulate a Pulsar Survey

Once you've generated a pulsar population model, the `dosurvey` script can be used to run the model through a past, present or future, pulsar survey (as specified in files in the `survey` directory shipped with the code .. `_model-survey-files`).

For example, say we want to take the population model we just created, `pop_ne2025.model`, and estimate from this how many pulsars would be detected in a putative LOFAR pulsar survey. This can be simply done using:

```
dosurvey -f pop_ne2025.model -surveys LOFAR
```

4.4 Using PsrPopPySuper as a Python Module

The `PsrPopPySuper` package can also be used as a Python module. The following example shows how to generate a population model, evolve it, and then run it through a survey, all in one script:

```
from psrpoppysuper import evolve, dosurvey

evolution = evolve.generate(nngen=1269,
    # surveyList=['PMPS_NEW'],
    age_max=1.0E+9,
    birthVModel='exp',
    birthVPars=[0, 380],
    electronModel='ne2001',
    bFieldPars=[12.35, 0.55],
    lumDistType='fk06',
    lumDistPars=[-1.5, 0.5],
    pDistPars=[0.3, 0.15],
    braking_index=3
)

survey = dosurvey.run(evolution, surveyList=['PMPS_NEW'])

dosurvey.write(survey, asc=True, extension=f'{i}.results')
```

These runs the `PsrpopPySuper` functions directly, and the output is a population object which can be used for further analysis or visualisation. The `write` function can be used to save the results to a file, with the option to save as an ascii file for easy inspection.

MODULE-LEVEL DOCUMENTATION

5.1 *pulsar* – Creates/stores a pulsar object

class pulsar.Pulsar

Pulsar.__init__([*period*, *dm*, *gl*, *gb*, *galCoords*, *r0*, *dtrue*, *lum_1400*, *spindex*, *alpha*, *rho*, *width_degree*, *snr*, *beaming*, *scindex*, *gpsFlag*, *gpsA*, *brokenFlag*, *brokenSI*])

Initialise the pulsar object

Pulsar.s_1400()

Returns the flux at 1400 MHz, calculated as

$$S_{1400} = \frac{L_{1400}}{D_{\text{true}}^2}$$

Pulsar.width_ms()

Returns the pulse width in milliseconds, calculated as

$$W_{\text{ms}} = P_{\text{ms}} \times W_{\text{degree}}/360$$

5.2 *population* – Creates/stores a population

class population.Population

Population.__init__([*pDistType*, *radialDistType*, *lumDistType*, *pmean*, *psigma*, *simean*, *sisigma*, *lummean*, *lumsigma*, *zscale*, *electronModel*, *gpsFrac*, *gpsA*, *brokenFrac*, *brokenSI*, *ref_freq*])

Initialise the population object

Population.__str__()

Defines how the operation print Population is performed

Population.size()

Returns the number of pulsars in the population object

Population.join(*poplist*)

Joins each of the populations in list *poplist* to the current population

Population.write(*outf*)

Uses cPickle to dump the population to file *outf*

Population.write_asc(*outf*)

Writes the population to an ascii file in the old psrpop way

5.3 *survey* – Read a survey file into a survey object

class `survey.Survey`

`Survey.__init__(surveyName)`

Read in a (correctly formatted!) survey file

`Survey.__str__()`

Define how to perform `print` `Survey`

`Survey.nchans()`

Returns the number of channels, calculated as

$$n_{\text{chans}} = \frac{BW_{\text{total}}}{BM_{\text{chan}}}$$

`Survey.inRegion(pulsar)`

Determines if *Pulsar* is inside survey region. Returns True or False accordingly

`Survey.inPointing(pulsar)`

Determines if *Pulsar* is inside one of the survey's *pointings*. Returns the offset from beam centre to the pulsar.

`Survey.SNRcalc(pulsar, pop)`

Calculates the SNR of a *Pulsar* from *Population* `pop` in the survey. Returns -1 if pulse is smeared, and -2 if pulsar is outside survey region. SNR is calculated (with familiar terms) as

$$\text{SNR} = \frac{S_{1400} G \sqrt{n_{\text{pol}} BW \tau}}{\beta T_{\text{tot}}} \sqrt{\frac{1-\delta}{\delta}} \eta$$

where

$$\eta = \exp(-2.7727 \times \text{offset}^2 / \text{fwhm}^2)$$

class `survey.Pointing`

`Pointing.__init__(coord1, coord2, coordtype)`

Converts (coord1, coord2) into correctly formatted (l, b). Coordtype must be either eq or gal. If eq, the RA and Dec are converted internally

5.4 *populate* – Create a population object

class `populate.Populate`

`Populate.generate(nngen[, surveyList, pDistType, radialDistType, electronModel, pDistPars, siDistPars, lumDistType, lumDistPars, zscale, duty, scindex, gpsArgs, doubleSpec, nostdout ])`

The method called by the `populate.py` command-line-script

`Populate.write(outf=populate.model)`

Writes the *Population* model into file `outf` as a cPickle dump

5.5 *radialmodels* – Container for radial distn models

class `radialmodels.RadialModels`

`radialmodels.seed()`

Call the FORTRAN routine to make a seed

`radialmodels.slabdist()`

Pick a point from a “slab” distribution around the Galactic plane

`radialmodels.diskdist()`

Pick a point from a distribution purely along the Galactic plane

`radialmodels.lfl06()`

Pick a point from the Lorimer et al (2006) Galactic distribution

`radialmodels.ykr()`

Pick a point from the Yusifov & Kucuk Galactic distribution

5.6 *galacticops* – Container for functions relating to the Galaxy

`class galacticops.GalacticOps`

`galacticops.calc_dtrue((x, y, z))`

Calculate the distance from the Sun to Galactic coords (x, y, z) (NB. tuple)

`galacticops.calcXY(r0)`

Given a Galactic radius r0, choose an (x, y) position at random θ

`galacticops.ne2025_dist_to_dm(dist, gl, gb)`

Given a distance and Galactic coordinates, calculate DM according to NE2025

`galacticops.ne2001_dist_to_dm(dist, gl, gb)`

Given a distance and Galactic coordinates, calculate DM according to NE2001

`galacticops.ymw16_dist_to_dm(dist, gl, gb)`

Given a distance and Galactic coordinates, calculate DM according to the YMW16 model

`galacticops.lb_to_radec(gl, gb)`

Convert Galactic coordinates to equatorial

`galacticops.ra_dec_to_lb(ra, dec)`

Convert equatorial coordinates to Galactic

`galacticops.tsky(gl, gb, freq)`

Calculate sky temperature at observing frequency freq and at Galactic coordinates gl, gb according to Haslam et al

`galacticops.xyz_to_lb((x, y, z))`

Convert the tuple (x, y, z) to Galactic sky coordinates.

Returns l, b in degrees

`galacticops.lb_to_xyz(l, b, dist)`

Convert Galactic sky coordinates at a distance dist to x,y,z coordinates.

Returns position as a tuple

`galacticops.scatter_bhat(dm, scatterindex, freq_mhz)`

Calculate the scatter time according to Bhat et al at. Frequency in MHz, pulsar with dispersion measure dm, and using a scattering spectral index of scatterindex.

Calculated as

$$\tau = -6.46 + 0.154 \log_{10}(\text{dm}) + 1.07 \log_{10}(\text{dm})^2 + \text{scatterindex} \times \log_{10}\left(\frac{\text{freq_mhz}}{1000}\right)$$

and typically scatterindex = -3.86 (but there is an option to vary it!)

5.7 *evolve* – Evolve a pulsar population object

class `evolve.EvolveException`

`evolve.generate`(*nngen*, *surveyList=None*, *age_max=1.0E9*, *pDistPars=[.3, .15]*, *bFieldPars=[12.65, 0.55]*,
birthVPars=[0.0, 180.], *siDistPars=[-1.6, 0.35]*, *alignModel='orthogonal'*,
lumDistType='fk06', *lumDistPars=[-1.5, 0.5]*, *alignTime=None*, *spinModel='fk06'*,
beamModel='tm98', *birthVModel='gaussian'*, *electronModel='ne2025'*, *braking_index=0*,
zscale=0.05, *duty=5.*, *scindex=-3.86*, *widthModel=None*, *nodeathline=False*,
efficiencycut=None, *nostdout=False*, *nospiralarms=False*, *keepdead=False*, *ascfile=None*)

Generate an evolved pulsar population model.

`evolve.main()`

Entry point for the evolve console script.

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

PYTHON MODULE INDEX

e

evolve, 16

g

galacticops, 15

p

populate, 14

population, 13

pulsar, 13

r

radialmodels, 14

s

survey, 14

Symbols

`__init__()` (*population.Population method*), 13
`__init__()` (*pulsar.Pulsar method*), 13
`__init__()` (*survey.Pointing method*), 14
`__init__()` (*survey.Survey method*), 14
`__str__()` (*population.Population method*), 13
`__str__()` (*survey.Survey method*), 14
`--asc`
 dosurvey.py command line option, 8
`--keepdead`
 evolve.py command line option, 10
`--logx`
 view.py command line option, 10
`--nodeathline`
 evolve.py command line option, 10
`--noresults`
 dosurvey.py command line option, 8
`--nospiralarms`
 evolve.py command line option, 10
`--nostdout`
 dosurvey.py command line option, 8
 evolve.py command line option, 10
 populate.py command line option, 8
`--summary`
 dosurvey.py command line option, 8
`-alignmodel`
 evolve.py command line option, 9
`-aligntime`
 evolve.py command line option, 9
`-asc`
 evolve.py command line option, 8
 populate.py command line option, 7
`-b`
 evolve.py command line option, 9
`-beammodel`
 evolve.py command line option, 9
`-bi`
 evolve.py command line option, 9
`-dm`
 evolve.py command line option, 8
 populate.py command line option, 7
`-doublespec`
 populate.py command line option, 8
`-eff`
 evolve.py command line option, 9
`-f`
 dosurvey.py command line option, 8
 view.py command line option, 10
 visualize.py command line option, 10
`-frac`
 visualize.py command line option, 10
`-gps`
 populate.py command line option, 7
`-l`
 evolve.py command line option, 9
`-ldist`
 evolve.py command line option, 9
`-n`
 evolve.py command line option, 8
 populate.py command line option, 7
`-o`
 evolve.py command line option, 8
 populate.py command line option, 7
`-p`
 evolve.py command line option, 8
 populate.py command line option, 7
 view.py command line option, 10
`-pdist`
 populate.py command line option, 7
`-rdist`
 populate.py command line option, 7
`-sc`
 evolve.py command line option, 9
 populate.py command line option, 7
`-si`
 evolve.py command line option, 9
 populate.py command line option, 7
`-spinmodel`
 evolve.py command line option, 9
`-surveys`
 dosurvey.py command line option, 8
 evolve.py command line option, 8
 populate.py command line option, 7
`-tmax`

evolve.py command line option, 8
-v
 evolve.py command line option, 9
-vmodel
 evolve.py command line option, 9
-w
 evolve.py command line option, 9
 populate.py command line option, 7
-wmod
 evolve.py command line option, 9
-z
 evolve.py command line option, 9
 populate.py command line option, 7

C

calc_dtrue() (*in module galacticops*), 15
calcXY() (*in module galacticops*), 15

D

diskdist() (*in module radialmodels*), 15
dosurvey.py command line option
 --asc, 8
 --noresults, 8
 --nostdout, 8
 --summary, 8
 -f, 8
 -surveys, 8

E

evolve
 module, 16
evolve.py command line option
 --keepdead, 10
 --nodeathline, 10
 --nospiralarms, 10
 --nostdout, 10
 -alignmodel, 9
 -aligntime, 9
 -asc, 8
 -b, 9
 -beammodel, 9
 -bi, 9
 -dm, 8
 -eff, 9
 -l, 9
 -ldist, 9
 -n, 8
 -o, 8
 -p, 8
 -sc, 9
 -si, 9
 -spinmodel, 9
 -surveys, 8
 -tmax, 8

 -v, 9
 -vmodel, 9
 -w, 9
 -wmod, 9
 -z, 9

EvolveException (*class in evolve*), 16

G

galacticops
 module, 15
GalacticOps (*class in galacticops*), 15
generate() (*in module evolve*), 16
generate() (*populate.Populate method*), 14

I

inPointing() (*survey.Survey method*), 14
inRegion() (*survey.Survey method*), 14

J

join() (*population.Population method*), 13

L

lb_to_radec() (*in module galacticops*), 15
lb_to_xyz() (*in module galacticops*), 15
lf106() (*in module radialmodels*), 15

M

main() (*in module evolve*), 16
module
 evolve, 16
 galacticops, 15
 populate, 14
 population, 13
 pulsar, 13
 radialmodels, 14
 survey, 14

N

nchans() (*survey.Survey method*), 14
ne2001_dist_to_dm() (*in module galacticops*), 15
ne2025_dist_to_dm() (*in module galacticops*), 15

P

Pointing (*class in survey*), 14
populate
 module, 14
Populate (*class in populate*), 14
populate.py command line option
 --nostdout, 8
 -asc, 7
 -dm, 7
 -doublespec, 8
 -gps, 7

- n, 7
- o, 7
- p, 7
- pdist, 7
- rdist, 7
- sc, 7
- si, 7
- surveys, 7
- w, 7
- z, 7

population

- module, 13

Population (*class in population*), 13

pulsar

- module, 13

Pulsar (*class in pulsar*), 13

R

ra_dec_to_lb() (*in module galacticops*), 15

radialmodels

- module, 14

RadialModels (*class in radialmodels*), 14

S

s_1400() (*pulsar.Pulsar method*), 13

scatter_bhat() (*in module galacticops*), 15

seed() (*in module radialmodels*), 14

size() (*population.Population method*), 13

slabdist() (*in module radialmodels*), 14

SNRcalc() (*survey.Survey method*), 14

survey

- module, 14

Survey (*class in survey*), 14

T

tsky() (*in module galacticops*), 15

V

view.py command line option

- logx, 10

- f, 10

- p, 10

visualize.py command line option

- f, 10

- frac, 10

W

width_ms() (*pulsar.Pulsar method*), 13

write() (*populate.Populate method*), 14

write() (*population.Population method*), 13

write_asc() (*population.Population method*), 13

X

xyz_to_lb() (*in module galacticops*), 15

Y

ykr() (*in module radialmodels*), 15

ymw16_dist_to_dm() (*in module galacticops*), 15